



# Practica N°1.

## Ejercicio 1.

**Materia:**

Bases de Datos

**Profesor:**

Gabriel Hurtado Avilés

**Grupo:**

3CV2

**Presentan:**

Ramirez González José Issac

Silva Anaya Ariana





# Control de versiones con Git y GitHub

## 1. Qué es un sistema de control de versiones y qué problema resuelve en un trabajo en equipo?

Un Sistema de Control de Versiones (VCS, por sus siglas en inglés) es una herramienta de software diseñada para registrar, gestionar y rastrear los cambios realizados sobre el código fuente o cualquier conjunto de archivos a lo largo del tiempo. Permite consultar un historial completo de modificaciones, revertir el proyecto a estados o revisiones previas y coordinar la intervención simultánea de múltiples desarrolladores sobre una misma base de código.

En la dinámica del trabajo en equipo, un VCS resuelve de forma directa el caos derivado de la edición concurrente. Sin una gestión formal de versiones, la colaboración se degrada rápidamente en dinámicas ineficientes como el intercambio manual de archivos comprimidos (ej. proyecto\_v1.zip, proyecto\_final\_modificado.zip), la sobreescritura accidental del trabajo de otros desarrolladores y la imposibilidad de determinar el origen de errores o regresiones en el sistema. El VCS resuelve estos problemas al centralizar el historial, aislar el trabajo en desarrollo de la versión estable de producción y permitir la fusión controlada de aportaciones individuales.

## 2. Diferencia entre Git y GitHub

A menudo se utilizan de manera indistinta, pero Git y GitHub no son sinónimos ni cumplen la misma función dentro del ecosistema de desarrollo de software:

Git: Es el motor local de control de versiones. Se trata de un software ejecutable por línea de comandos o interfaz gráfica que opera de manera descentralizada en la





máquina personal del desarrollador. Administra la creación de ramas, el guardado de estados (commits) y la resolución de fusiones localmente, sin depender de una conexión a internet.

GitHub: Es una plataforma en la nube que hospeda repositorios Git remotos. Ofrece una interfaz web e infraestructura colaborativa construida sobre Git, añadiendo herramientas como gestión de usuarios y permisos, revisión de código mediante Pull Requests, seguimiento de incidencias (issues), automatización e integración continua (CI/CD).

| Criterio          | Git                                                     | GitHub                                                   |
|-------------------|---------------------------------------------------------|----------------------------------------------------------|
| Naturaleza        | Herramienta de control de versiones distribuido (Local) | Plataforma SaaS de alojamiento web en la nube (Remoto)   |
| Ejecución         | Se ejecuta en la computadora local del usuario          | Se ejecuta en los servidores remotos de GitHub/Microsoft |
| Función principal | Registrar cambios, gestionar ramas y resolver fusiones  | Colaboración en equipo, revisión de código y CI/CD       |

### 3. Definición de Conceptos Clave con Ejemplos Prácticos

- Repositorio (Repository): Espacio de almacenamiento digital estructurado donde reside todo el código fuente de un proyecto, sus archivos de configuración y el registro histórico completo de sus cambios.

Ejemplo práctico: La carpeta raíz de un sistema e-commerce que contiene el código backend, frontend, scripts de base de datos e historial de Git.

- Confirmación (Commit): Una instantánea o captura del estado actual de los archivos rastreados en un punto específico del tiempo. Representa una unidad discreta y permanente de cambio acompañada de un mensaje explicativo.





Ejemplo práctico: Ejecutar ``git commit -m "Fix: corregir validación de correo en el formulario de registro"` para guardar la solución a un fallo específico.

- Rama (Branch): Una línea de desarrollo independiente que diverge de la línea principal (usualmente ``main`` o ``master``). Permite trabajar en funcionalidades o correcciones de forma aislada sin afectar el entorno estable.

Ejemplo práctico: Crear la rama ``feature/pasarela-pagos`` para desarrollar e integrar la API de Stripe sin alterar el código que se encuentra operando en producción.

- Fusión (Merge): Operación que combina el historial de confirmaciones y los cambios de una rama secundaria dentro de una rama de destino.

Ejemplo práctico: Unir la rama probada ``feature/pasarela-pagos`` hacia la rama principal ``main`` para hacer disponible la nueva función a todos los usuarios.

- Conflicto de fusión (Merge Conflict): Situación en la que Git no puede combinar automáticamente dos ramas porque se realizaron modificaciones incompatibles en las mismas líneas de un archivo en ambas ramas.

Ejemplo práctico: Si el Desarrollador A modifica la línea 20 de ``app.js`` en la rama A y el Desarrollador B modifica la misma línea 20 en la rama B, Git detiene el merge e instruye a resolver manualmente cuál versión prevalecerá.

- Pull Request (PR): Una solicitud formal dentro de una plataforma como GitHub para fusionar los cambios de una rama de trabajo hacia una rama base. Funciona como un espacio de discusión y revisión técnica.

Ejemplo práctico: Abrir un PR desde ``feature/login-social`` hacia ``dev`` solicitando la revisión de código por parte de los desarrolladores sénior antes de su integración.

- Archivo `.gitignore`: Archivo de configuración en texto plano que especifica patrones de nombres de archivos y directorios que Git debe omitir intencionalmente y no incluir en el control de versiones.





Ejemplo práctico: Agregar ``node_modules/``, ``.env`` (con claves secretas) y carpetas de compilación ``/dist`` al archivo ``.gitignore`` para evitar subir archivos pesados o sensibles al repositorio.

- Archivo README: Documento de bienvenida y referencia principal en la raíz del repositorio, redactado comúnmente en formato Markdown (`README.md``). Expone el propósito del proyecto, requisitos de entorno, guía de instalación y normas de contribución.

Ejemplo práctico: Un archivo `README.md`` que explica las instrucciones paso a paso para clonar el repositorio, ejecutar ``npm install`` y levantar el servidor localmente con ``npm start``.

#### **4. Flujo de Trabajo Basado en Ramas y Revisión Entre Pares**

Un flujo de trabajo basado en ramas (como GitHub Flow o Git Flow) establece la regla técnica de que la rama principal (usualmente `main`` o `production``) debe mantenerse en todo momento limpia, estable y lista para desplegarse a producción. Bajo esta metodología, ningún desarrollador escribe ni confirma código directamente sobre la rama principal. Cualquier desarrollo se realiza en una rama corta de característica (feature branch) y solo se incorpora mediante un Pull Request tras pasar las validaciones requeridas.

¿Por qué el código se revisa entre pares (Code Review) antes de fusionarse?

Aseguramiento de la Calidad y Prevención de Errores: Permite identificar fallos de lógica, vulnerabilidades de seguridad, edge cases no considerados y cuellos de botella de rendimiento antes de que el código afecte el entorno de producción.

Mantenimiento de Estándares de Arquitectura y Estilo: Garantiza que todo el equipo respete las mismas convenciones de código, patrones de diseño y niveles de documentación, manteniendo la mantenibilidad del sistema a largo plazo.





Transferencia de Conocimiento y Aprendizaje Continuo: Evita los silos de información donde un solo desarrollador conoce una parte crítica del sistema. Fomenta el aprendizaje recíproco entre desarrolladores junior y senior.

Responsabilidad Colectiva sobre el Producto: El código deja de ser propiedad individual para convertirse en un activo colectivo del equipo de ingeniería. Si ocurre un fallo en producción, la responsabilidad es compartida por quienes construyeron y revisaron la solución.

## Referencias

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. Disponible en acceso abierto: <https://git-scm.com/book/en/v2>

GitHub Docs. (2026). About Pull Requests and Code Review. GitHub Documentation. <https://docs.github.com/en/pull-requests>

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson. (Capítulo 25: Gestión de Configuración y Control de Versiones).

